

✦ JOSA //

Performance, Profiling & Debugging

OpenLab Apprenticeship 2026

Week 08

// 1 //

Learning Goals

- Read a flame graph and identify hot paths.
- Diagnose the most common performance bug in real systems: the **N+1 query**.
- Apply the **USE method** (Utilization / Saturation / Errors) and the **RED method** (Rate / Errors / Duration) to system-level diagnosis.
- Know your one debugger well. Set breakpoints, inspect state, step through code.

// 2 //

Technical Deep Dive

// 2.1 The first rule of performance work



SOME QUOTE :)

"We should forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil. Yet we should not pass up our opportunities in that critical 3%." — Donald Knuth

Most of your code doesn't matter for speed (the 97%). Only a small part (the 3%); the bits that run over and over (in a loop, on every request, on every user) actually determines how fast your program feels. The rest could be sloppy and nobody would notice.

So the rule is: **don't optimize based on gut feeling**. Follow this process instead:

1. **Measure first.** Run a profiler before changing anything. Your instinct about what's "probably slow" is usually wrong. Profilers exist because human intuition about performance is unreliable.
2. **Fix what actually costs the most, not what looks slowest.** A function that runs once at startup barely matters, even if you speed it up 10x. A function that runs on every single request matters a lot, even a small speedup there helps every user, every time. Prioritize by *total impact* (how often it runs × how slow it is), not by how impressive the fix sounds.

```
def start_server():                # runs ONCE – total cost is irrelevant
    load_config()
    connect_to_database()

def handle_request(req):          # runs THOUSANDS of times per minute
    data = fetch_user(req)        # This is probably in the 3%
    return render(data)
```



IMPORTANT

Even if `start_server()` is sloppy and takes an extra 200ms, nobody cares as it happens once. If `handle_request()` is 5ms slower than it needs to be, that's 5ms × every request, forever. That's where the real cost lives.

- 3. Measure again after fixing.** Don't assume your fix worked. Sometimes a change that should help actually makes things worse elsewhere. Trust the numbers, not your assumption that you fixed it.

// 2.2 Flame graphs

Flame graphs (invented by Brendan Gregg) are the go-to way to visualize where your program is spending its time. They look confusing at first, but the idea is simple once you know what the axes mean:

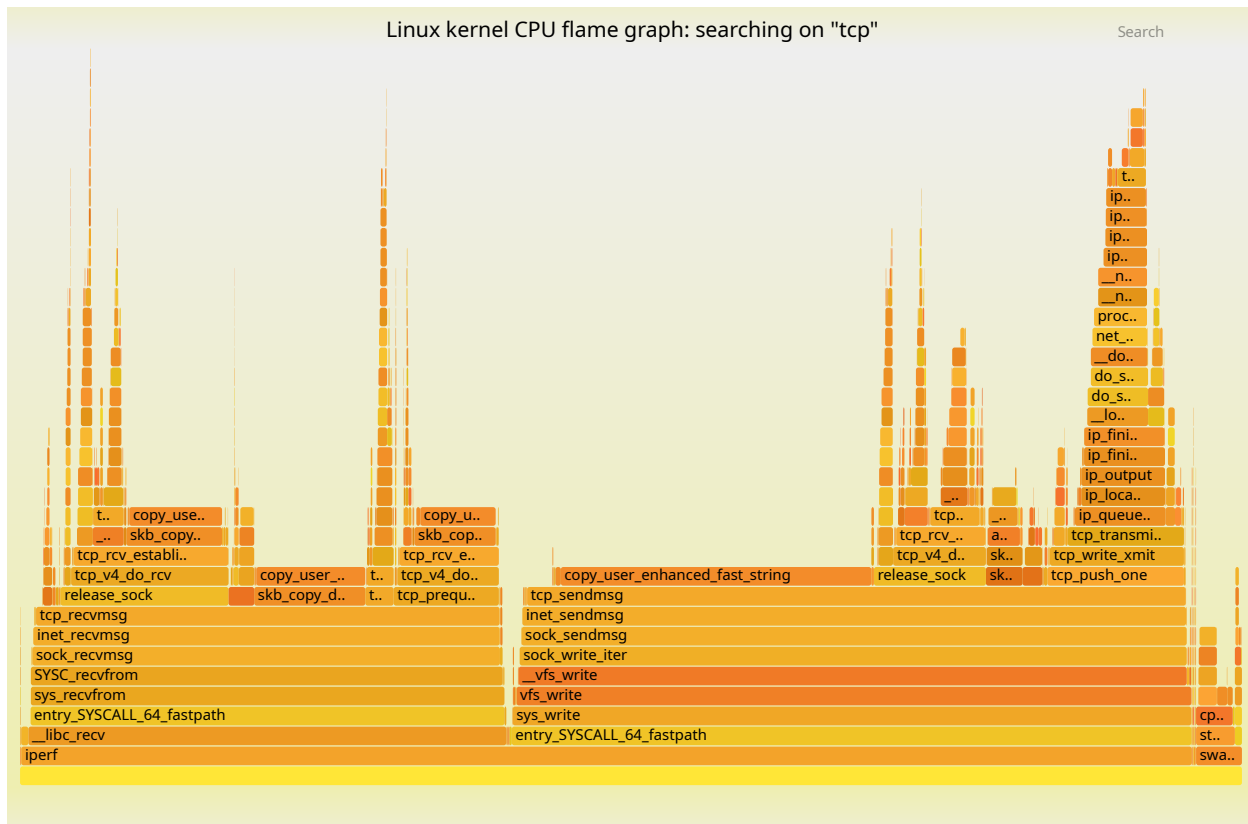
- **The x-axis is NOT time.** It just shows how often a function showed up while the profiler was sampling. Width = how much CPU time was spent inside that function (including everything it called).
- **The y-axis is call depth.** The bottom is your program's entry point. Every layer up is a function called by the layer below it (function → the function it called → the function *that* called, and so on).



THE TAKEAWAY

So basically: wider = more CPU time spent there. Taller = deeper into nested function calls.

Reading a flame graph



- **Wide block at the top of a stack**

This is the *last* function in the chain – the one actually doing work, not just handing off to someone else. If it's wide, it means: this specific function, by itself, is eating CPU time.

This is your prime suspect. Go optimize this function directly.

- **A stack that stays wide as it goes up, then suddenly narrows**

Picture a chain: A calls B calls C calls D. If A, B, and C are all wide, but D is suddenly skinny – that means most of the time was already spent *before* D even got called. So the real cost isn't D, it's whatever A/B/C were doing before D. **The point where the width drops is where the time actually went.** Don't blame the top of the stack – look at the last spot it was still wide.

- **A skinny sliver going way up high**

This is a function that got called, but hardly ever. So even though it went "deep," it took basically no total time. **Ignore it**, unless you're specifically chasing rare, occasional slowness like that 1-in-100 slow request (a p99 spike) rather than everyday performance.



NOTE

A "p99 spike" means: normally your slowest 1% of requests are, say, 200ms but suddenly they jump to 3 seconds. Most users (the 99%) don't notice anything wrong. But a small, real slice of users are having a bad time and since it's rare and inconsistent, it's genuinely hard to catch with casual testing. You need monitoring/tracing to even see it's happening.

One-line summary: Width = time cost, height = how deep the call chain goes. Find the widest blocks – that's where your time is actually going.

// 2.3 Per-language profilers

Language	Profiler	What it does
Python	py-spy	Watches a running Python program from outside and records where time is spent, then outputs a flame graph (profile.svg)
Python	scalene	Similar, but also tells you which lines are eating <i>memory</i> , not just CPU
Node.js	Chrome DevTools	You start Node in a special "inspect" mode, then plug the Chrome browser's dev tools into it to watch performance
Node.js	clinic	A tool that runs your app and produces a performance report/diagnosis automatically
Go	pprof	Go's built-in profiler, run your tests with profiling on, then open the results in a browser-based UI
Rust	cargo-flamegraph	Runs your Rust program and directly outputs a flame graph
C/C++	perf (Linux)	A Linux system tool, records what the program is doing at the CPU level, then shows a report

py-spy gets a special callout: most profilers require you to *start* the program with profiling turned on (you plan ahead). py-spy is different. It can attach to a program that's **already running**, live, in production, without restarting it or changing any code. That's a big deal because:

- You don't need to reproduce the slowness in a test environment
- You don't risk restarting a production service just to investigate it
- You can literally point it at a real, currently-slow process and watch what it's doing right now

The example command:

```
py-spy record -o profile.svg --pid 12345 --duration 30
```

Means: "attach to process ID 12345, watch it for 30 seconds, save what you found as a flame graph called profile.svg."

Practical takeaway: find your language, and that's your go-to profiler and the command to start with.

// 2.4 The N+1 query, the most common real-world bottleneck

Imagine a blog page that shows the 50 latest posts, along with each post's author name. A naive way to build this:

```
posts = db.query("SELECT * FROM posts ORDER BY date DESC LIMIT 50")
for post in posts:
    post.author = db.query("SELECT * FROM users WHERE id = ?", post.author_id) # 50 queries
return render(posts)
```

Here's the trap: to get the 50 posts, that's 1 query. But then, for *each* post, it goes back to the database *again* to look up who wrote it. So instead of one efficient trip, you make **51 separate trips to the database** (1 + 50 – hence "N+1", where N = number of posts).

- **Why this is bad**

Each database query has some fixed cost, say 5ms just for the round trip. 51 queries × 5ms = **255ms**, before you've even started building the actual page. And this gets *worse* as your data grows, 500 posts would mean 501 queries. This pattern quietly kills performance at scale even though it "works fine" with small test data, which is exactly why it sneaks into production.

- **The fix**

Instead of asking the database 51 separate times, ask it *once*, telling it to fetch the posts and their authors together (using a JOIN, which combines data from two tables in a single query). Most frameworks have a built-in way to say "get this and also automatically get its related data in one go". That's called **eager loading** (in Django, Rails, SQLAlchemy, and Hibernate it might be called `select_related`, `includes`, `selectinload`, or JOIN FETCH, depending on the framework).

- **Why it's sneaky**

You won't see this bug by just looking at the page as it loads fine, looks fine. You only catch it by looking *underneath* the page, at the actual list of SQL queries being run (via SQL logs or APM/monitoring tools). Nearly every backend developer accidentally ships this at least once, because it's invisible unless you specifically go looking for it.



THE TAKEAWAY

At some point during development, turn on SQL logging and look at a real page with real traffic. If you see the same query pattern repeating dozens of times, that's an N+1 – and it's fixable with one line in most frameworks.

// 2.5 USE and RED: two checklists for "something's slow, but why?"

When you don't know where a slowdown is coming from, don't just poke around randomly. Use one of these two checklists depending on what you're looking at.

USE method : For hardware/system resources (CPU, memory, disk, network)

For each resource, ask three questions:

- **Utilization:** How busy is it? (e.g., "CPU is at 95%" or "disk is at 80% busy")
- **Saturation:** Is work piling up waiting for it? (e.g., a queue of tasks waiting for the CPU to free up)
- **Errors:** Is it actually failing? (e.g., dropped network packets, failed disk reads)

Go through CPU, memory, disk, and network one by one, checking all three. Whichever resource shows a problem first (very high utilization, a backed-up queue, or actual errors) – that's your bottleneck.

RED method: for your own services/APIs

For a service you own, track:

- **Rate:** How many requests per second is it handling?
- **Errors:** How many of those are failing, per second? .
- **Duration:** How long are requests taking? (Look at the *distribution* – p50/p95/p99 – not just the average, since averages hide the slow outliers, as covered earlier.)

The rule of thumb

- Watching **your own app/service** → use **RED** (is it handling traffic well, is it erroring, is it fast).
- Watching **the machine/infrastructure underneath** (the server, the database box, the network) → use **USE** (is a resource maxed out, backed up, or failing).

Learning Resources

Topic	Resource
Flame graphs	brendangregg.com/flaregraphs.html
Systems performance methodology	Systems Performance (2nd ed.) – Brendan Gregg
USENIX ATC '17: Visualizing Performance with Flame Graphs – Brendan Gregg	https://www.youtube.com/watch?v=D53T1Ejig1Q
USE method	USE Method – brendangregg.com
Debugging zines	Julia Evans – Wizard Zines
Python sampling profiler	py-spy
Memory + line-level Python	scalene
Node.js	Clinic.js
Go pprof	go.dev/blog/pprof
Rust	cargo-flamegraph
Record-replay debugger	rr-project.org
Tracing (modern)	eBPF – ebpf.io

Suggested Repositories

- [benfred/py-spy](#) – Active, friendly project; profiler internals are educational reading even if you don't contribute.
- **Any of your own slow code.** Profile something you wrote. Find one bottleneck. Fix it. Document the before/after.
- [plasma-umass/scalene](#) – Active research-led project at UMass.

// 5 //

Your Tasks for the Week

- 1. Profile and fix (on someone else's repo).** Pick an open-source project you actually use, a library, CLI tool, or small app. Profile a real workflow in it (running its test suite, a CLI command, a script using the library). Find a function that takes >50ms and identify the bottleneck with a flame graph. If you can produce a small, safe fix, open a PR. Either way, open an issue on the project with your findings with flame graph attached, the specific function, and the numbers (e.g. "840ms for 10k items, function X is 91% of that time"). Document the before/after in your journal regardless of whether the maintainers respond.
- 2. Find an N+1.** Pick an open-source web app, turn on SQL logging, and browse around for a page issuing N+1 queries. This one's a bonus. N+1s aren't always easy to spot from the outside, and not every app you pick will have one. If you find one: open an issue describing it (with the query log as evidence), and if the fix is small and obvious (an eager load / JOIN), open a PR. **If you don't find one, that's fine, note it in your journal and move on.**
- 3. Read 3 real flame graphs.** Find any 3 flame graphs published online (e.g., on brendangregg.com) and write about it in your journal in your own words on what each one says. What's hot, what's the bottleneck.
- 4. Use a real debugger.** Pick a debugger for your primary language ([pdb](#), Chrome DevTools, [dlv](#), [lldb](#)). Solve one bug end-to-end with breakpoints. No print statements. Document the experience.

// 6 //

Soft Skill of the Week: Intellectual Humility

Performance work is humbling. Your intuition will be wrong. Code you thought was fast will be slow; code you wrote off as "probably fine" will turn out to dominate the profile. The senior move is **trusting the measurement over the story**.

// 6.1 Three Habits

1. **State a hypothesis before you measure.** “I think the bottleneck is the database call.” Then measure. If you were wrong, notice. Over months, calibrate yourself. Engineers who do this become good; engineers who skip it stay surprised forever.
2. **When you’re wrong, say so out loud.** “I thought X but the data shows Y.” This is also the most credible form of leadership – admitting error in public is a flex, not a weakness.
3. **Keep a “wrong predictions” journal.** Every time you confidently predicted something and were wrong, write one line. Re-read it monthly. You’ll start spotting the categories where your gut systematically fails (and the ones where it’s right).

**TIP**

There’s a related skill: **distinguishing what you know from what you assume**. In a debug session, the assumptions you don’t check are where the bug hides. “The function is being called” is an assumption. “I added a print and saw it print” is knowledge.